Class 6: R functions

Laura (PID: A17844089)

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call the function.
- Input **arguments**, there can be multiple comma seperated inputs to the function.
- The **body**, lines of R code that do the work of the function.

Our first wee function:

```
add <- function(x, y=1) {
 x + y
}
```

Lets test our function

```
add(10, 10)
```

```
[1] 20
```

```
add(10)
```

```
[1] 11
```

A second function

Let's try something more interesting. Make a sequence generation tool.

The `sample()` function could be useful here.

```
sample(1:10, size = 3)
```

```
[1] 2 5 7
```

change this to work with the nucleotides A C G and T and return 3 of them

```
n <- c("A", "C", "G", "T")
sample(n, size=15, replace = TRUE)
```

```
 [1] "A" "T" "A" "G" "G" "A" "G" "G" "G" "A" "A" "T" "G" "C" "C"
```

Turn this snipet into a function that returns a user specified length dna sequence. Let's call it `generate_dna`...

```
generate_dna <- function(len=10, fasta=FALSE) {
  n <- c("A", "C", "G", "T")
  v <- sample(n, size=len, replace = TRUE)

  # Make a single element vector
  s <- paste(v, collapse="")

  cat("Well done you!\n")

  if(fasta) {
    return( s )
  } else {
    return( v )
   }
  }
```

```
generate_dna(5)
```

```
Well done you!

[1] "C" "T" "G" "C" "A"
```

```
s <- generate_dna(15)
```

```
Well done you!
```

```
s
```

```
 [1] "T" "A" "C" "C" "T" "G" "A" "C" "A" "T" "C" "C" "C" "C" "C"
```

I want the option to return a single element character vector with my sequence all together like this: "GGAGTAC"

```
s
```

```
 [1] "T" "A" "C" "C" "T" "G" "A" "C" "A" "T" "C" "C" "C" "C" "C"
```

```
paste(s, collapse="")
```

```
[1] "TACCTGACATCCCCC"
```

I want the option to return a single element character vector with my sequence all together like this: "GGAGTAC"

```
generate_dna(10, fasta=FALSE)
```

```
Well done you!
```

```
 [1] "T" "T" "G" "A" "A" "G" "G" "G" "G" "G"
```

```
generate_dna(10, fasta=TRUE)
```

```
Well done you!
```

```
[1] "AGCTGACTAG"
```

A more advanced example

Make a third function that generates protein sequence of a user specified length and format.

```
generate_protein <- function(size=15, fasta=TRUE) {
  aa <- c("A", "R", "N", "D", "C", "Q", "E",
          "G", "H", "I", "L", "K", "M", "F",
          "P", "S", "T", "W", "Y", "V")

  seq <- sample(aa, size = size, replace = TRUE)

  if(fasta) {
    return(paste(seq, collapse = ""))
  } else {
    return(seq)
   }
}
```

Try this out...

```
generate_protein(10)
```

```
[1] "QITLYNSHVR"
```

Q. Generate random protein sequences between lengths 5 and 12 amino acids

```
generate_protein(5)
```

```
[1] "TVFKA"
```

```
generate_protein(6)
```

```
[1] "QQGVSI"
```

One approach is to do this by brute force calling our function for each length 5 to 12.

Another approach is to write a `for()` loop to iterate over the input valued 5 to 12

A very useful third R specific approach is to use the `sapply()` function.

```
seq_lengths <- 6:12
for (i in seq_lengths) {
  cat(">", i, "\n")
  cat( generate_protein(i) )
  cat("\n")
}
```

```
> 6
SKYDGD
> 7
DTHWKAC
> 8
HQSDELYE
> 9
WHVATAEIA
> 10
SPTLRRKHFF
> 11
FYISAPHCMTD
> 12
DMGIKPEYKYDR
```

```r
sapply(5:12, generate_protein)
```

```
[1] "EFYGV"        "PKAEEQ"       "SLTTRES"      "SKGFHCWD"     "QTKAGENAL"
[6] "AAKFSEPLHQ"   "YDWMSVTQAID"  "MLFMNAPPQCMM"
```

> **Key-Point**: Writing functions in R is doable but not the easiest thing. Starting with a working snippet of code and then using LLM tools to improve and generalize your function code is a productive approach.